慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/04/30)

04 - 単語をベクトルで表現する

情報工学科 准教授　高道 慎之介

Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	**2026/04/30**	**単語をベクトルで表現する**	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習１	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	
第13回	2026/07/09	自然言語処理から音声言語処理へ	おまけ
第14回	2026/07/16	演習２	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：線形代数・確率統計・微分積分

単語列で考えてみよう

私 / **は** / リンゴ / を / 食べる
彼 / **は** / バナナ / が / 好き / だ
私 / **は** / 公園 / に / 行く
リンゴ / が / 落ちる
私 / **は** / 赤い / リンゴ / を / 買う
空 / **は** / 青い
私 / **は** / 彼 / と / リンゴ / を / 分けた
彼女 / **は** / メロン / を / 食べる
私 / **は** / リンゴ / が / 大好き / だ
犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私}, \text{は}) = \underbrace{P(\text{は}|\text{私})}_{\text{“私” の後に “は” が出現する確率}}\underbrace{P(\text{私})}_{\text{“私” の出現確率}}$$

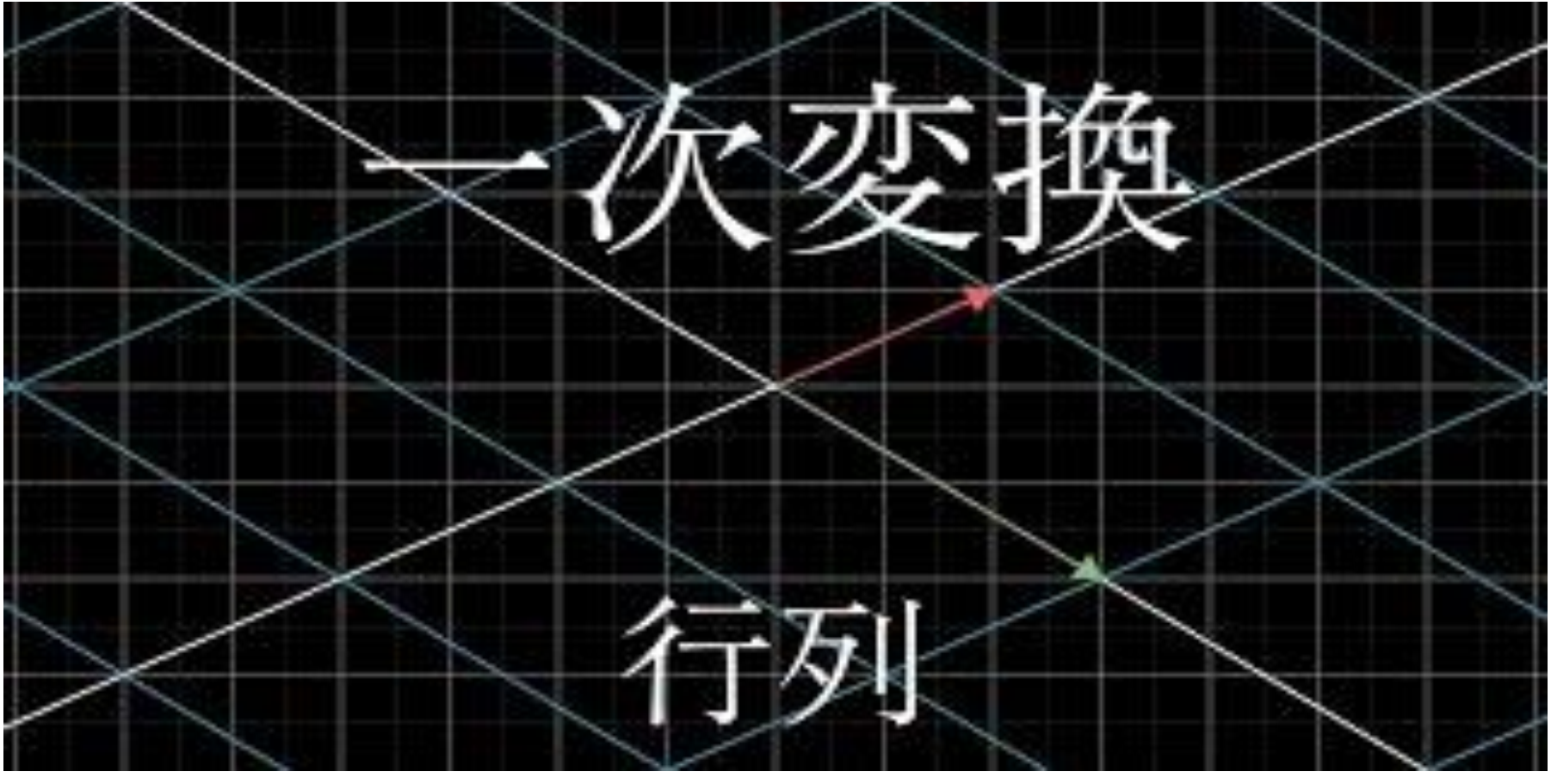
$$P(\text{私}) = \frac{5}{50}$$ (50 単語のうち 5 回出現)

$$P(\text{は}|\text{私}) = \frac{5}{5}$$ (“私” に後続する 5 単語のうち 5 回出現)

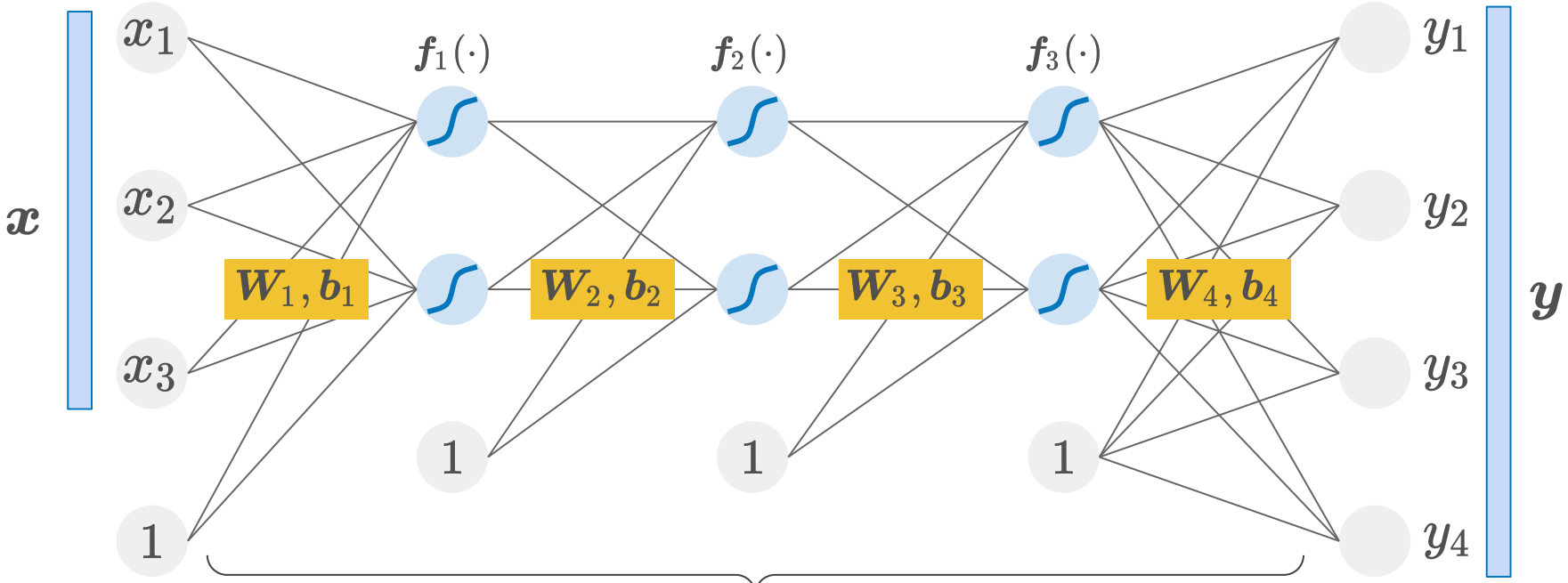
連続 3 単語以上にも拡張可能

$$P(\text{私}, \text{は}, \text{彼}) = P(\text{彼}|\text{は}, \text{私})P(\text{は}|\text{私})P(\text{私})$$

線形変換 $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ とアフィン変換



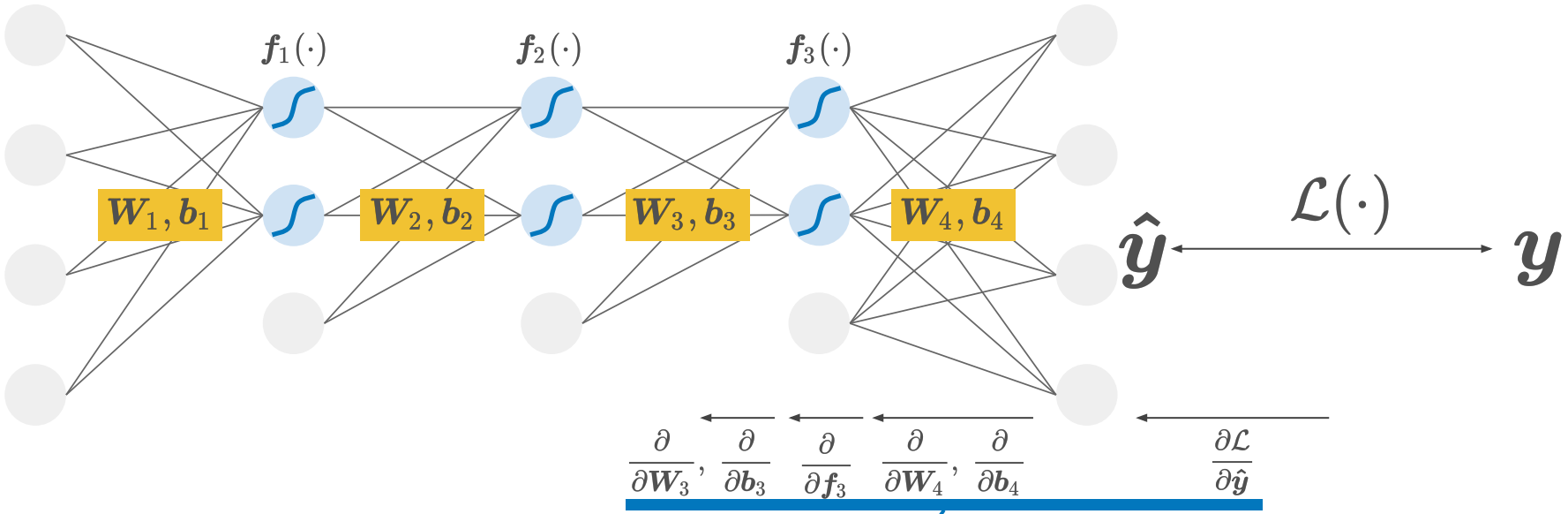
多層パーセプトロン



$\boldsymbol{y} = f_{\theta}(\boldsymbol{x})$ と表記

$\boldsymbol{W}_*, \boldsymbol{b}_*$ の集合 (パラメータθをもつ関数)

バックプロパゲーション

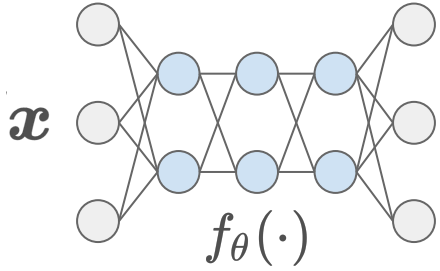


連鎖律で計算可能

$$W_3 \leftarrow W_3 - \alpha \frac{\partial \mathcal{L}}{\partial W_3}$$

更新後　更新前　目的関数の負の勾配

ニューラルネットワークを
離散確率変数の確率分布を出力するモデルだと考える



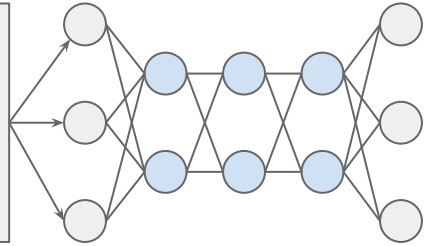
$p_\theta(\boldsymbol{y}|\boldsymbol{x})$

モデルパラメータ θ で計算される，
x に対する y の条件付き確率

“私”

単語を数値で
表現する
何らかの方法

次回以降に解説



$P(\text{は}|\text{私})$

$P(\text{を}|\text{私})$

$P(\text{に}|\text{私})$

単語列で考えてみよう

私 / は / リンゴ / を / 食べる
彼 / は / バナナ / が / 好き / だ
私 / は / 公園 / に / 行く
リンゴ / が / 落ちる
私 / は / 赤い / リンゴ / を / 買う
空 / は / 青い
私 / は / 彼 / と / リンゴ / を / 分けた
彼女 / は / メロン / を / 食べる
私 / は / リンゴ / が / 大好き / だ
犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私}, \text{は}) = \underbrace{P(\text{は}|\text{私})}_{\text{“私”の後に“は”が出現する確率}}\underbrace{P(\text{私})}_{\text{“私”の出現確率}}$$

$P(\text{私}) = \frac{5}{50}$ (50 単語のうち 5 回出現)

$P(\text{は}|\text{私}) = \frac{5}{5}$ (“私” に後続する 5 単語のうち 5 回出現)

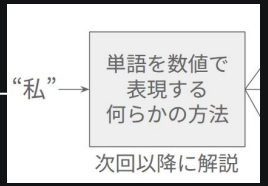
連続 3 単語以上にも拡張可能

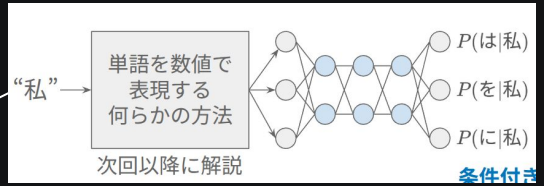
$$P(\text{私}, \text{は}, \text{彼}) = P(\text{彼}|\text{は}, \text{私})P(\text{は}|\text{私})P(\text{私})$$

条件付き確率を計算する，
パラメータ Θ を持つニューラルネット

以前の課題：条件付き確率を計算するニューラルネットワークを構築せよ

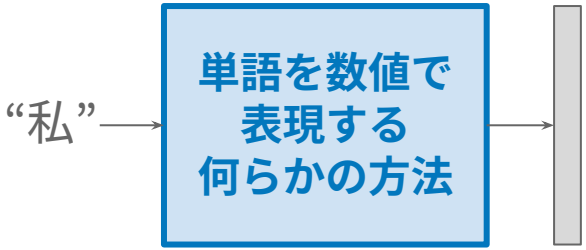
```python
# "あ", "い", "う" のいずれかを入力とし、
# 次元数 3 の確率値を出力するニューラルネットワークを構築してください
# ニューラルネットワークの構成，使用する Python モジュールは問いません.
# 学習は不要で，重みは適当に（例えば乱数値に）設定してください.
# 例：入力 "あ" に対して，出力 [0.7, 0.2, 0.1] を得る.
# すでに書かれている箇所は変更しても構いません.

input_vec = {
    "あ": [1, 0, 0],
    "い": [0, 1, 0],
    "う": [0, 0, 1]
}

def neural_network(input_char):
    """ニューラルネットワークの関数"""

    return output_vec # 3次元の確率値を返す


if __name__ == "__main__":
    """メイン関数"""

    output_a = neural_network("あ")
    output_i = neural_network("い")
    output_u = neural_network("う")

    print("入力: あ, 出力:", output_a) # 全要素は非負の値で，合計は 1 でなければならない
    print("入力: い, 出力:", output_i)
```





学習は不要．すなわち，
“あ” “い” “う” の入力から
確率値の条件を満たす
何らかの値が出ればOK

今日の目的：
単語の数値表現を学習する



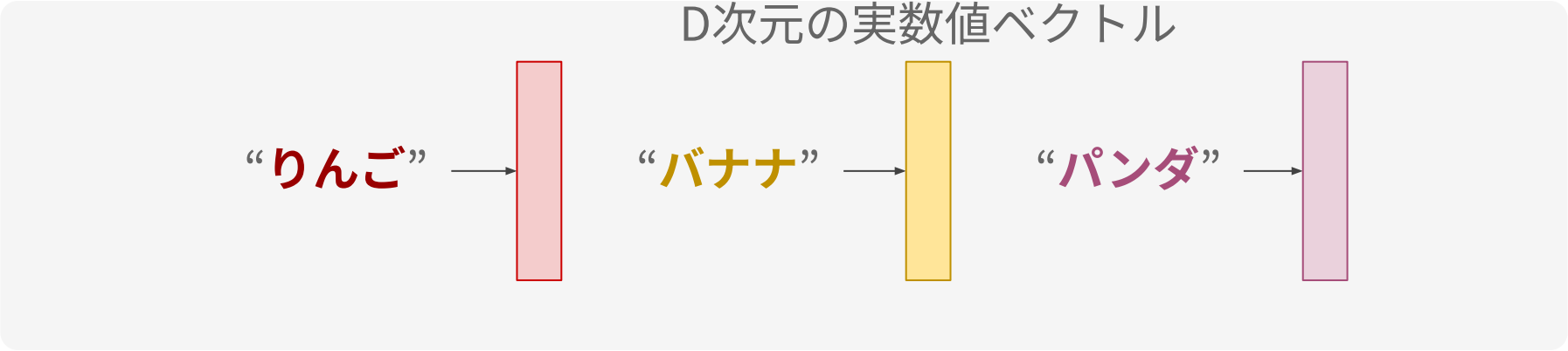
単語埋め込み (word embedding)

単語を数値表現するにはどうしたらいいだろう？

単語を計算機で扱うには単語を数値表現する

計算機は文字列をそのまま処理できないため，数値を扱うベクトル空間への写像が必要となる．

- 単語の集合 (語彙)：$V = \{\text{りんご}, \text{バナナ}, \text{パンダ}\}$
- 写像：$f : V \to \mathbb{R}^D$



もっとも簡単な方法は one-hot ベクトルで表すこと

次元数＝語彙数であり、特定要素のみが 1 でそれ以外は 0 のベクトル

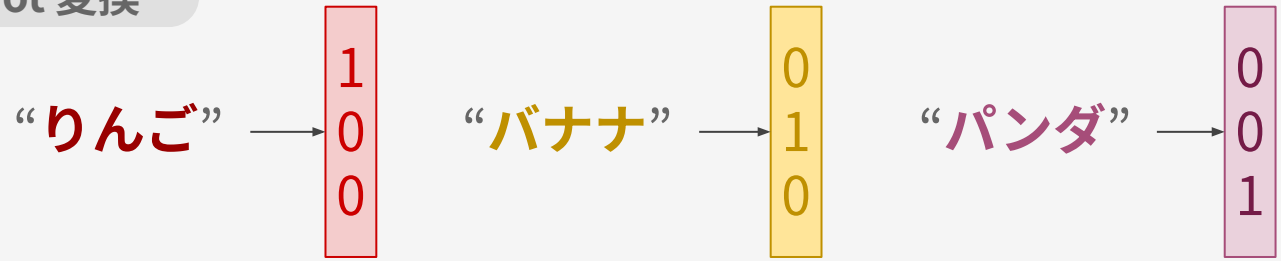
- 単語の集合 (語彙)：$V = \{w_1, \ldots, w_i, \ldots, w_{|V|}\}$
- 写像：$f : V \to \{0,1\}^{|V|}$

集合の大きさ＝語彙数

各要素が 0 or 1 の |V| 次元ベクトル

$$w_i \longmapsto \boldsymbol{x}_i = [x_1, \ldots, x_i, \ldots, x_{|V|}]^\top, x_j = \begin{cases} 1 & j = i \\ 0 & j \neq i \end{cases}$$

one-hot 変換



$V = \{$りんご, バナナ, パンダ$\}$ の場合

しかし one-hot ベクトル表現には問題がある

次元数の爆発

$$f : V \to \{0,1\}^{|V|}$$

語彙数 V に比例して次元数が増加.

新語や未知語によって語彙は際限なく増えるため、計算コストが膨大になる.

類似性を捉えられない

全ベクトルが互いに直交するため，意味的な近さを距離で表現できない.

「りんご」と「バナナ」の距離は、「りんご」と「パンダ」の距離と同じになってしまう。

one-hot 変換

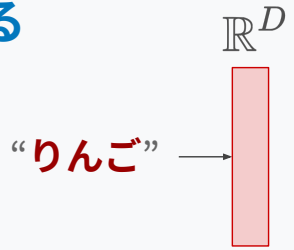
“りんご” → $\begin{bmatrix}1\\0\\0\end{bmatrix}$　“バナナ” → $\begin{bmatrix}0\\1\\0\end{bmatrix}$　“パンダ” → $\begin{bmatrix}0\\0\\1\end{bmatrix}$

$V = \{\text{りんご}, \text{バナナ}, \text{パンダ}\}$ の場合

代わりに，密な実数値ベクトルで表すことを考える
→ 単語埋め込み (word embedding)

あらかじめ定めた次元数 D の密ベクトル（＝埋め込み）に写像する

- 単語の集合 (語彙) : $V = \{w_1, \ldots, w_i, \ldots, w_{|V|}\}$
- 写像 : $f : V \to \mathbb{R}^D$



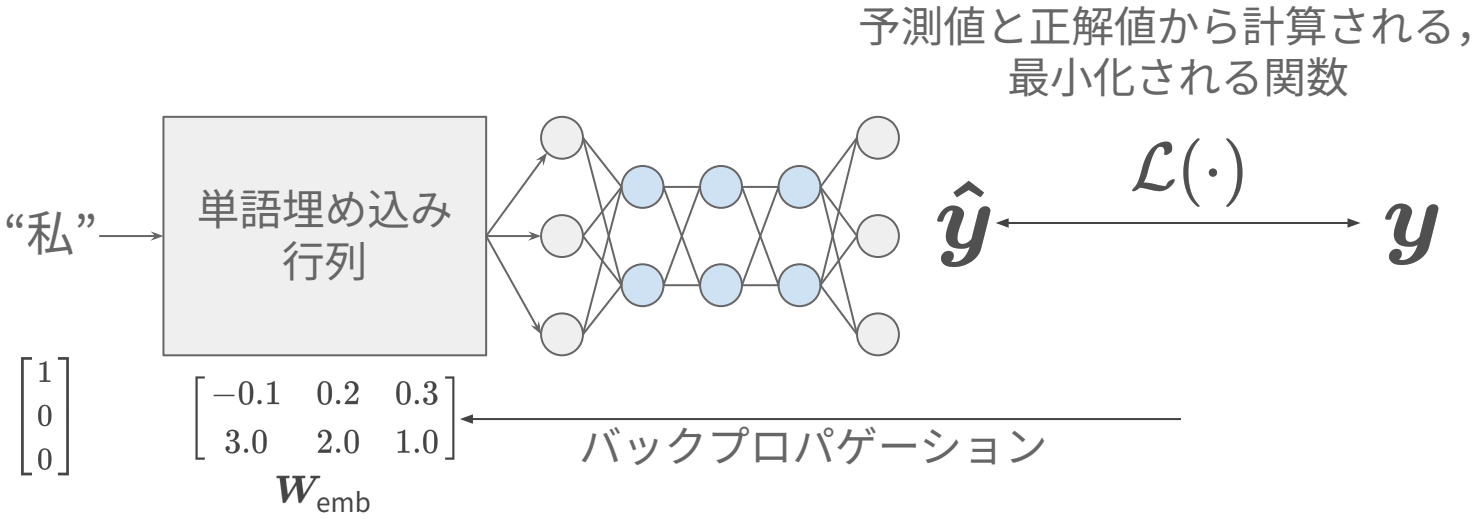
単語埋め込み行列による写像 (lookup)

実質的に行列の特定行の抽出．いわゆる Word Embedding 層．

$$\underbrace{\begin{bmatrix} -0.1 \\ 3.0 \end{bmatrix}}_{\mathbb{R}^D} = \underbrace{\begin{bmatrix} -0.1 & 0.2 & 0.3 \\ 3.0 & 2.0 & 1.0 \end{bmatrix}}_{\mathbb{R}^{D \times |V|}} \underbrace{\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}}_{\text{one-hot}} \text{“りんご”}$$

* one-hot ベクトルの次元数爆発は、この時点では解決できていない

単語埋め込み行列の学習＝単語埋め込みの学習



$$\boldsymbol{W}_{\text{emb}} \leftarrow \boldsymbol{W}_{\text{emb}} - \alpha \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}_{\text{emb}}}$$

ニューラルネットワークと同様に学習可能

単語埋め込みを得られると何ができる？

単語同士の類似度計算

- 埋め込み同士のコサイン類似度

$$\mathrm{cossim}(\boldsymbol{v}_a, \boldsymbol{v}_b) = \boldsymbol{v}_a^\top \boldsymbol{v}_b / (||\boldsymbol{v}_a||\ ||\boldsymbol{v}_b||)$$

- 類似度の比較

$$\mathrm{cossim}(\boldsymbol{v}_{りんご}, \boldsymbol{v}_{バナナ}) > \mathrm{cossim}(\boldsymbol{v}_{りんご}, \boldsymbol{v}_{パンダ})$$

- 類似した単語の検索

$$\mathrm{argmax}_{\boldsymbol{v} \in \mathcal{S}} \mathrm{cossim}(\boldsymbol{v}_{りんご}, \boldsymbol{v})$$

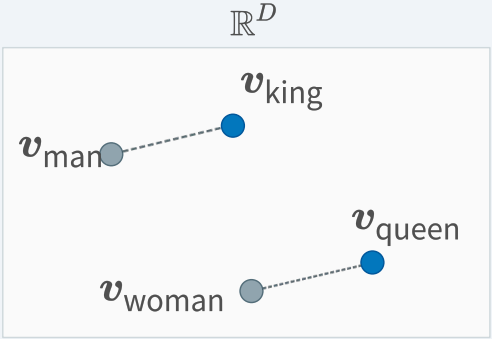
アナロジー (類推) の計算

- 意味の類推：王 - 男 + 女 = 女王

$$\boldsymbol{v}_{\text{queen}} \sim \boldsymbol{v}_{\text{king}} - \boldsymbol{v}_{\text{man}} + \boldsymbol{v}_{\text{woman}}$$

- 文法の類推：現在形 ⇄ 進行形

$$\boldsymbol{v}_{\text{flying}} \sim \boldsymbol{v}_{\text{dancing}} - \boldsymbol{v}_{\text{dance}} + \boldsymbol{v}_{\text{fly}}$$



Word2Vec

ニューラルネットワークを用いて埋め込みを学習する，最初期 (2013年) の方法

分布仮説 (distributional hypothesis)

“同じ文脈で現れる単語は同じ意味を持つ傾向にある” [Harris, 1954]

単語が持つ意味は，**周りの環境 (共起語)** を用いて説明できる

I saw **a** **girl** **with** red shoes

I saw **a** **lady** **with** red shoes

{girl, lady} は似た意味の単語

1つ前に **a**、1つ後に **with** が共起する
という同じ環境を持つため

Harris, Z. S. (1954). Distributional structure. Word, 10, 146–162.

Word2Vec 概観 [Mikolov13]

単語埋め込みを文から学習する最初期の技術．
分布仮説に基づいて，隣接して共起する単語を用いて学習する．

Skip-gram

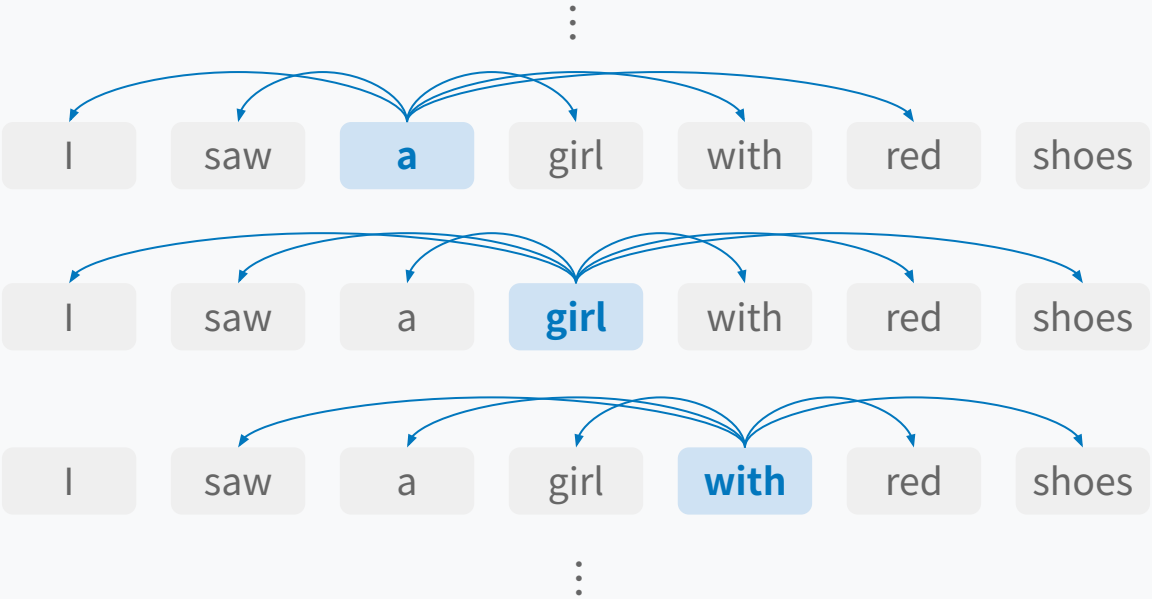
当該単語から隣接単語を予測

CBOW

隣接単語から**当該単語**を予測(本講義では省略)

Skip-gram の学習プロセス

分布仮説に基づいた学習：各単語の前後 M 単語（本例では M=3）を予測する．
これをコーパス全体のすべての文・すべての単語について繰り返す
＝データセットのこと



学習時に最小化する目的関数

Skip-gram の目的関数（負の対数尤度の最小化）

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-M\le j\le M, j\ne 0}\log P(w_{t+j}|w_t)$$

文章中のすべての単語について計算

前後 M 単語について計算（当該単語自身は除く）

Softmax による単語の予測確率

当該単語 w_t'との内積

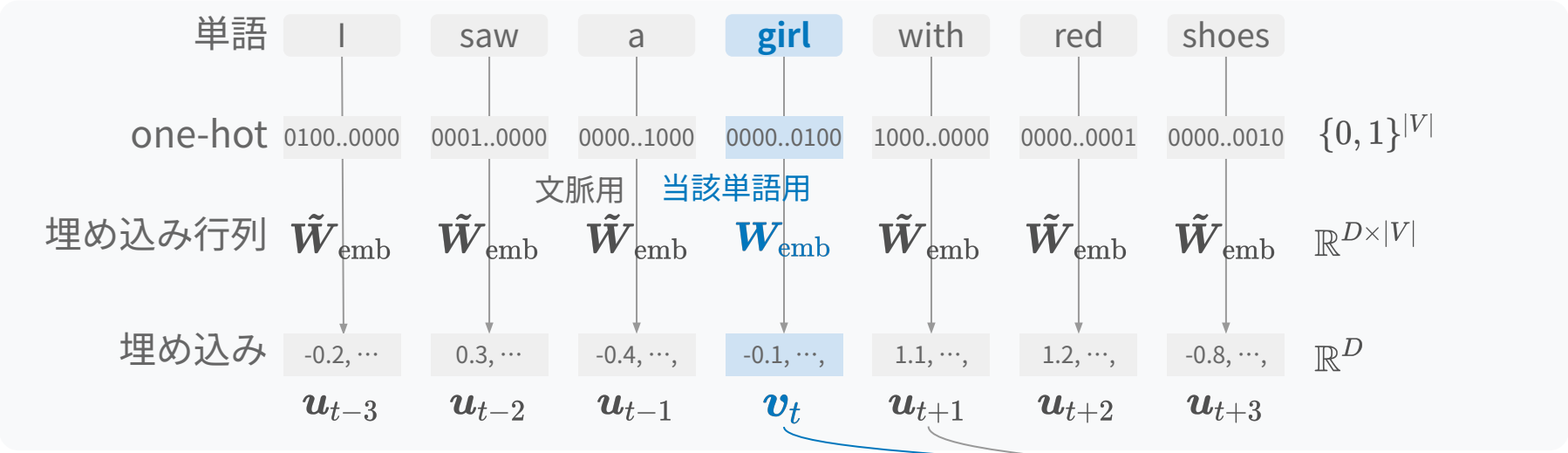
$$P(w_{t'}|w_t) = \frac{\exp(\boldsymbol{u}_{w_{t'}}^{\top}\boldsymbol{v}_{w_t})}{\sum_{w}^{V}\exp(\boldsymbol{u}_{w}^{\top}\boldsymbol{v}_{w_t})}$$

語彙 V 全体との内積の指数和

- 当該単語から隣接単語 (文脈) を予測できるように学習
  → 単語の距離 (内積) が近づくように学習

- 当該単語用と文脈 (前後の単語) 用で、異なる単語埋め込み行列を用いる
  - $\boldsymbol{v}_*$: 当該単語用の埋め込み行列から得る
  - $\boldsymbol{u}_*$: 文脈用の埋め込み行列から得る

Skip-gram モデルの計算イメージ



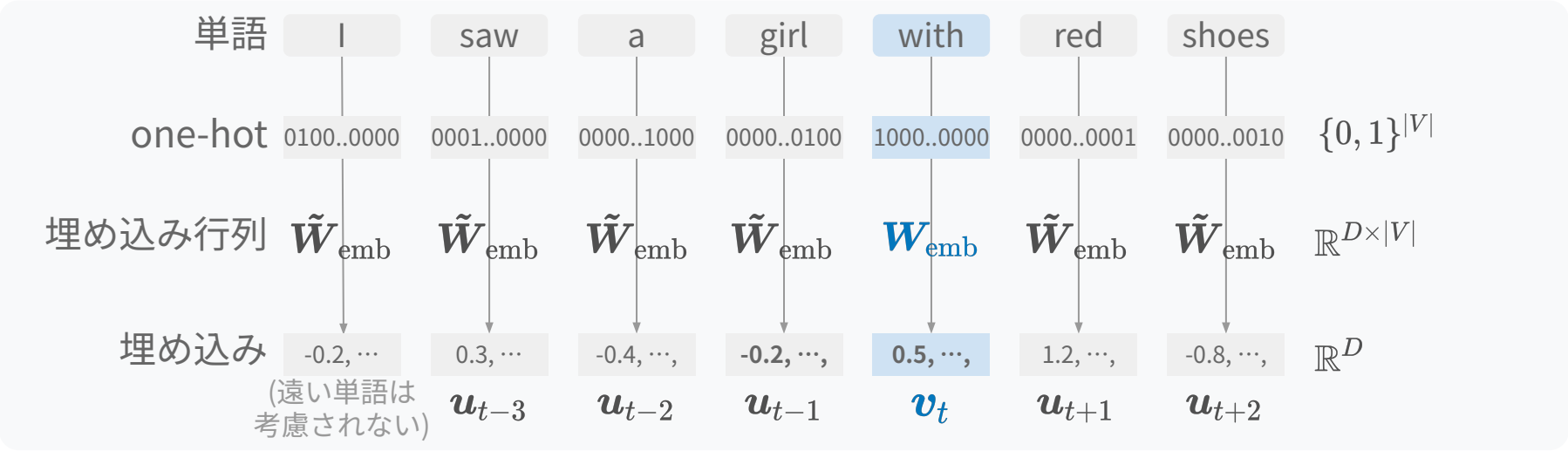
$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-M\le j\le M, j\neq 0}\log P(w_{t+j}|w_t)$$

$$P(w_{t'}|w_t) = \frac{\exp(\boldsymbol{u}_{w_{t'}}^{\top}\boldsymbol{v}_{w_t})}{\sum_{w}^{V}\exp(\boldsymbol{u}_{w}^{\top}\boldsymbol{v}_{w_t})}$$

語彙 V に含まれる
すべての単語との内積

Skip-gram の目的関数（負の対数尤度の最小化）

Skip-gram モデルの計算イメージ (スライドしても同じ)



- コーパスに含まれるすべての文についてこれを繰り返し、行列を更新
  → 学習後には、当該単語用の行列 W_emb を使って単語埋め込みを得る

ナイーブな実装の問題点と解決法

【問題点】計算コストが膨大

推論のたびに全語彙 V との内積計算が必要となり、非常に効率が悪い.

$$P(w_{t'}|w_t) = \frac{\exp(\boldsymbol{u}_{w_{t'}}^\top \boldsymbol{v}_{w_t})}{\sum_{w}^{V} \exp(\boldsymbol{u}_w^\top \boldsymbol{v}_{w_t})}$$

語彙数 (単語数) が 10000 なら，1個の当該単語の確率を計算するときに 10000 個の埋め込みが必要

【解決策】負例サンプリング

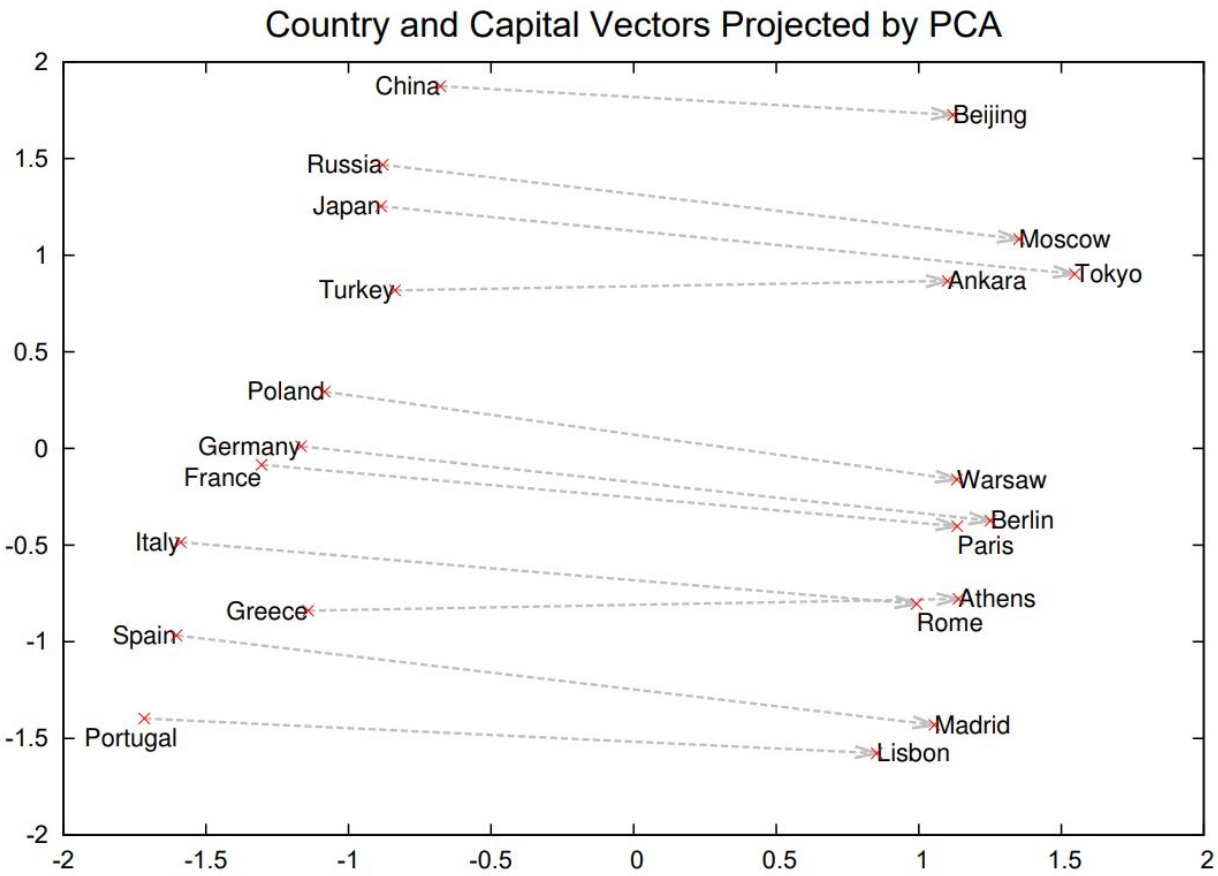
全語彙ではなく、確率的にサンプリングした一部の負例単語のみを用いて学習.

サンプリングされた k 個の負例を使用

$$V \Rightarrow \{w_1, \ldots, w_k\}$$

Skip-gram の目的関数における計算量を削減し、大規模なコーパスでの学習を可能にする.

単語埋め込み結果を PCA したもの



Word2vec の改良：FastText

Word2vec の課題

未知語への対応

- 語彙にない単語はすべて共通の "未知語" ベクトルに集約され、個別の意味が失われる.

複合語・派生語の無視

- "information" と "informatics" を全く別の独立した単語として扱う。
- 部分文字列が持つ共通の意味情報を活用できない.

FastText のアプローチ

サブワード (部分単語) 単位のベクトル化

- 単語を 3〜6 文字程度の文字集合（n-gram）に分割.

```
where -> {^wh, whe, her, ere, re$, ^where$}
```

単語ベクトルの生成

- 単語全体のベクトル = 各サブワードベクトルの総和

$$\boldsymbol{v}_{\text{where}} = \sum_{w \in \{\text{^wh, whe, her, ere, re\$, \^where\$}\}} \boldsymbol{v}_w$$

- 未知語でも部分的な共通点から意味を推測可能.

Piotr Bojanowski et al. Enriching Word Vectors with Subword Information. TACL.

Skip-gram を Pytorch で実装する

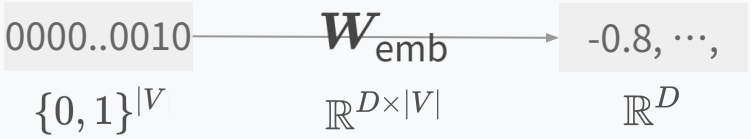
(Pytorch について最低限は説明するが，詳細は各自勉強しておくこと)

実装を理解するために最低限覚えておくこと

torch.nn.Embedding

```
torch.nn.Embedding(
    num_embeddings, embedding_dim)
```

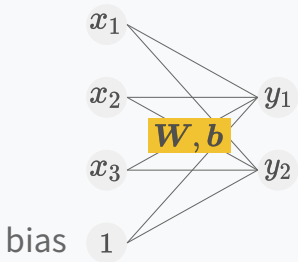
$|V|$ D



torch.nn.Linear

```
torch.nn.Linear(
    in_features, out_features, bias=True)
```

x の次元数　　y の次元数　　バイアスを使う



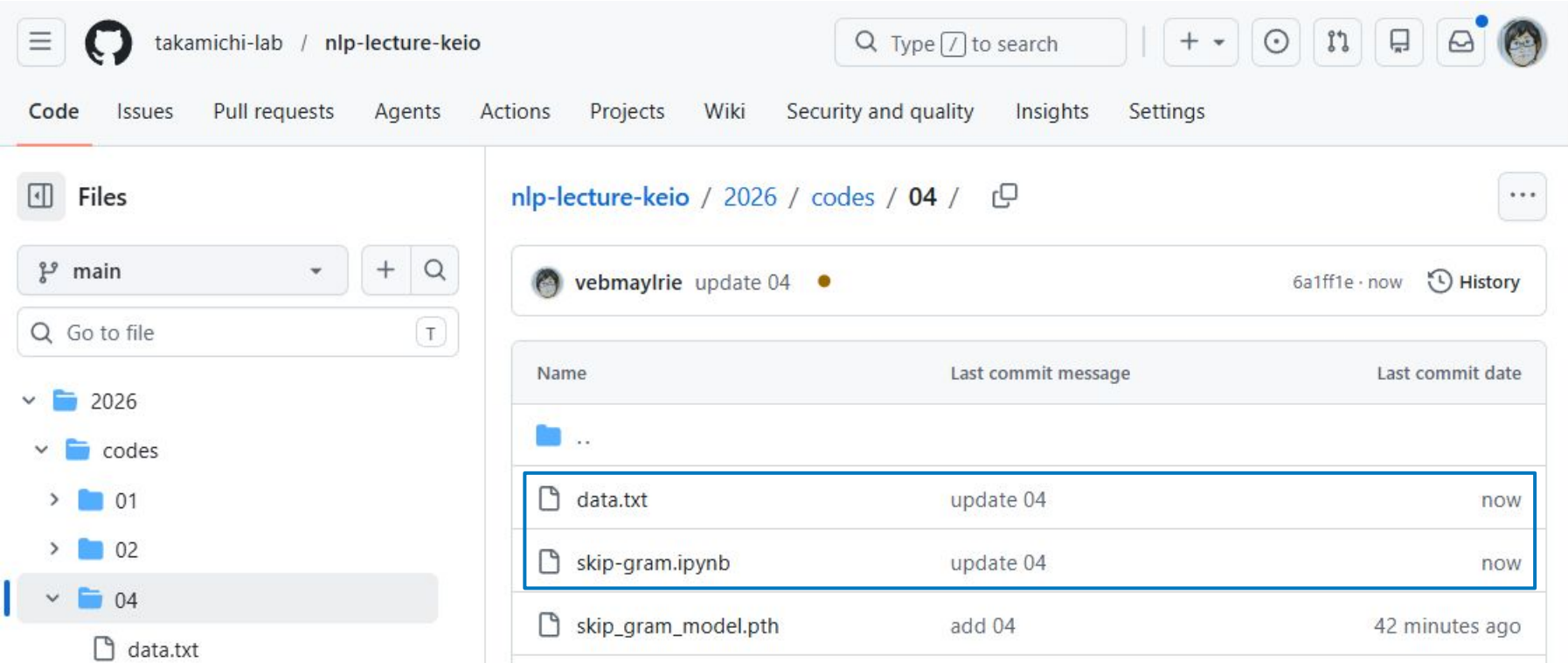
collections.defaultdict

```
collections.defaultdict(
    default_factory=None)
```

```
scores = defaultdict(lambda: -100) #存在しないキーに対する処理を定義

scores["Alice"] = 90

print(f"Alice's score: {scores['Alice']}")  # 90
print(f"Charlie's score: {scores['Charlie']}")# -100
```

実装してみよう on Google Colab

https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/04



サブワード (部分単語)

LLM 時代においては、サブワード単位が主流

基本的な考え方：頻度の高くない単語を分割すること

the companies are expanding → the **compan _ies** are **expand _ing**

サブワード化のメリット

学習パラメータの共有

複合語の効率化

- 通常銃猟 → 通常 **_銃猟**
- 緊急銃猟 → 緊急 **_銃猟**

活用語の効率化

- 美しい → **美し** _い
- 美しく → **美し** _く

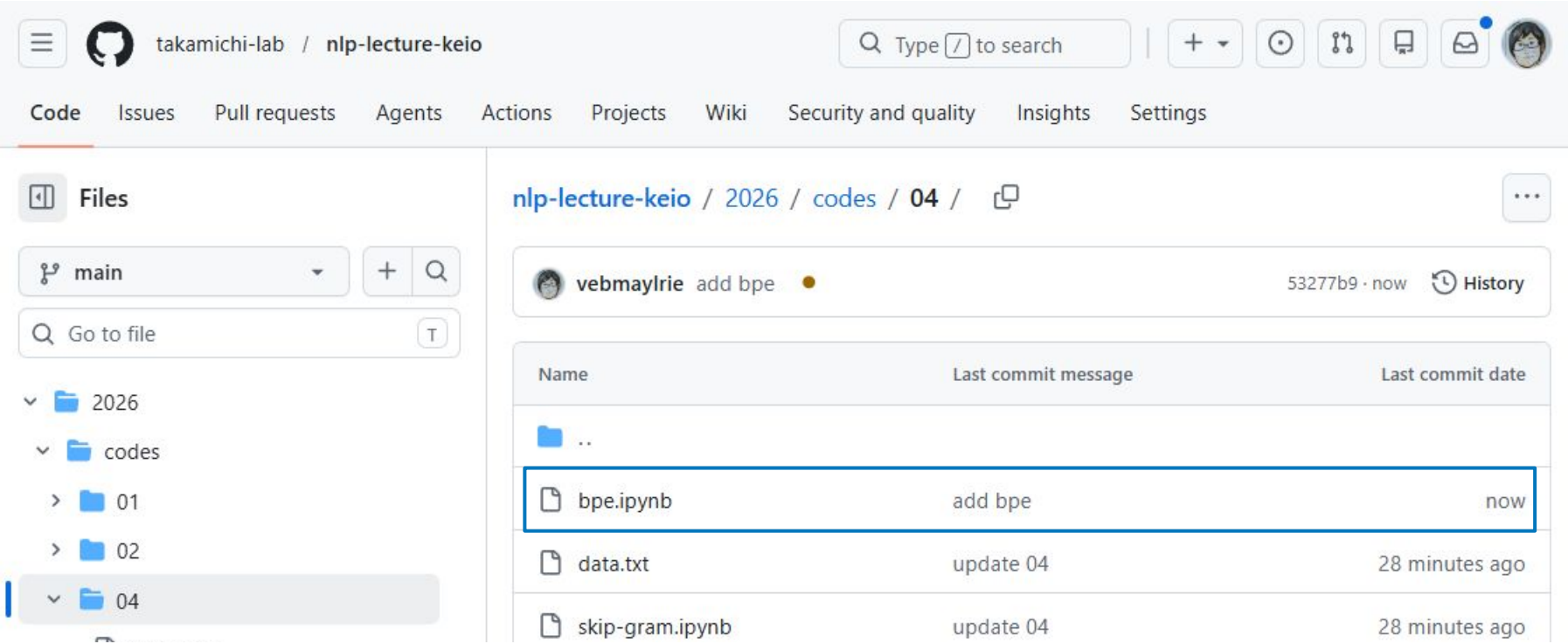
計算コストの削減と未知語の解消

- 単語単位よりも語彙数を大幅に削減
- 未知語も部分単語に分解するため，未知の入力が存在しなくなる．

Byte pair encoding (BPE) を実践してみよう

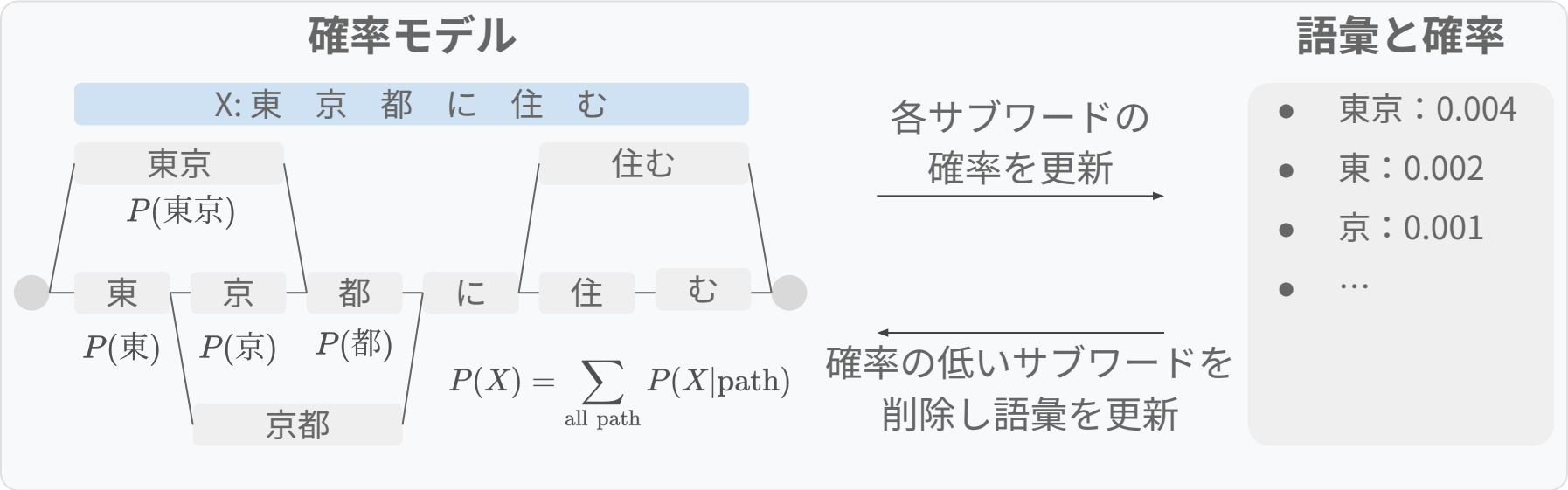
https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/04

頻度の高い文字列を逐次的に連結していく



1-gram 言語モデルに基づく方法 (SentencePiece に搭載)

1-gram 確率が最も高くなる語彙と出現確率を求める．EM アルゴリズムに基づく．

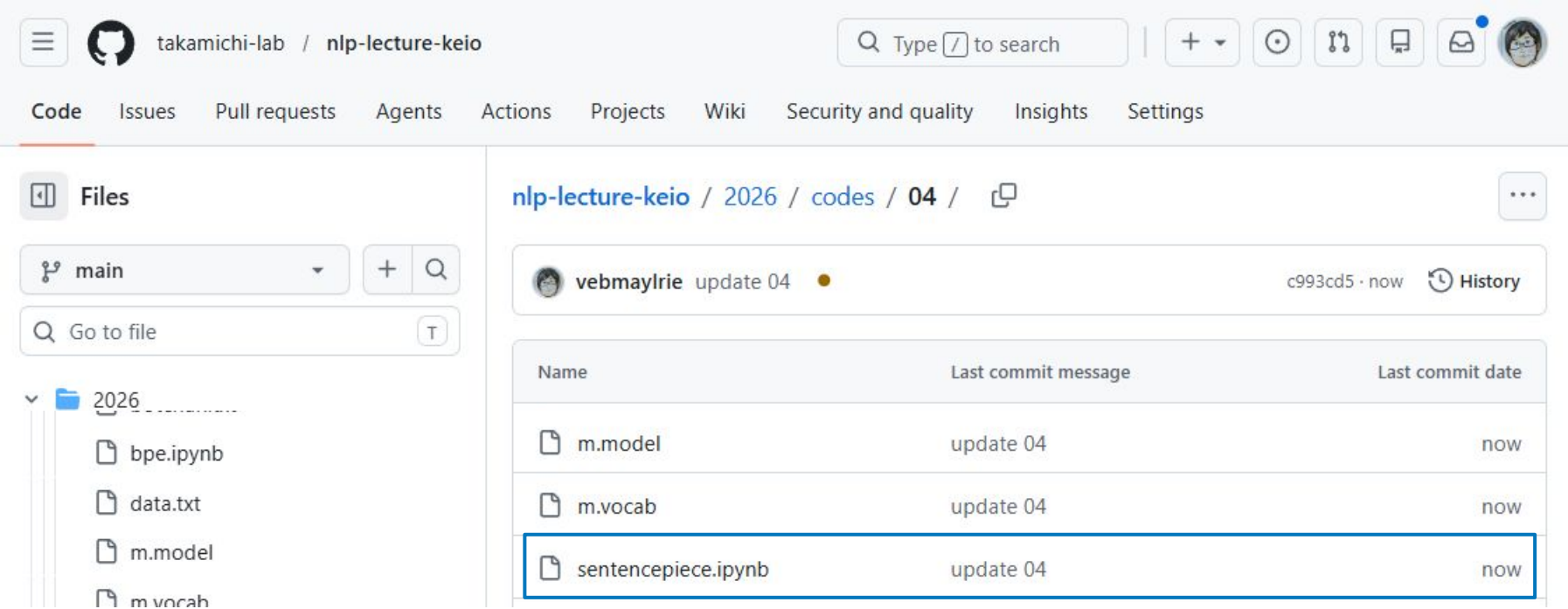


- **言語非依存**：言語固有の知識を必要としない汎用的な処理構造。
- **推論アルゴリズム**：1-gram 確率の総和が最大となる境界（Viterbi 等）で分割

SentencePiece を実践してみよう

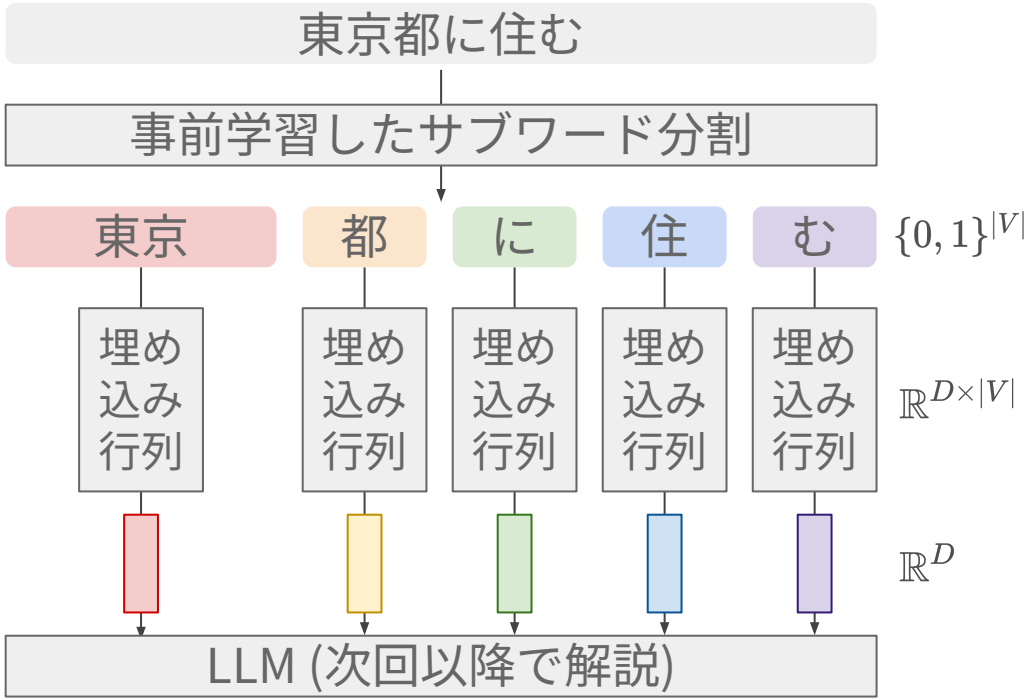
https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/04

1-gram 確率の低いの文字列を削っていく



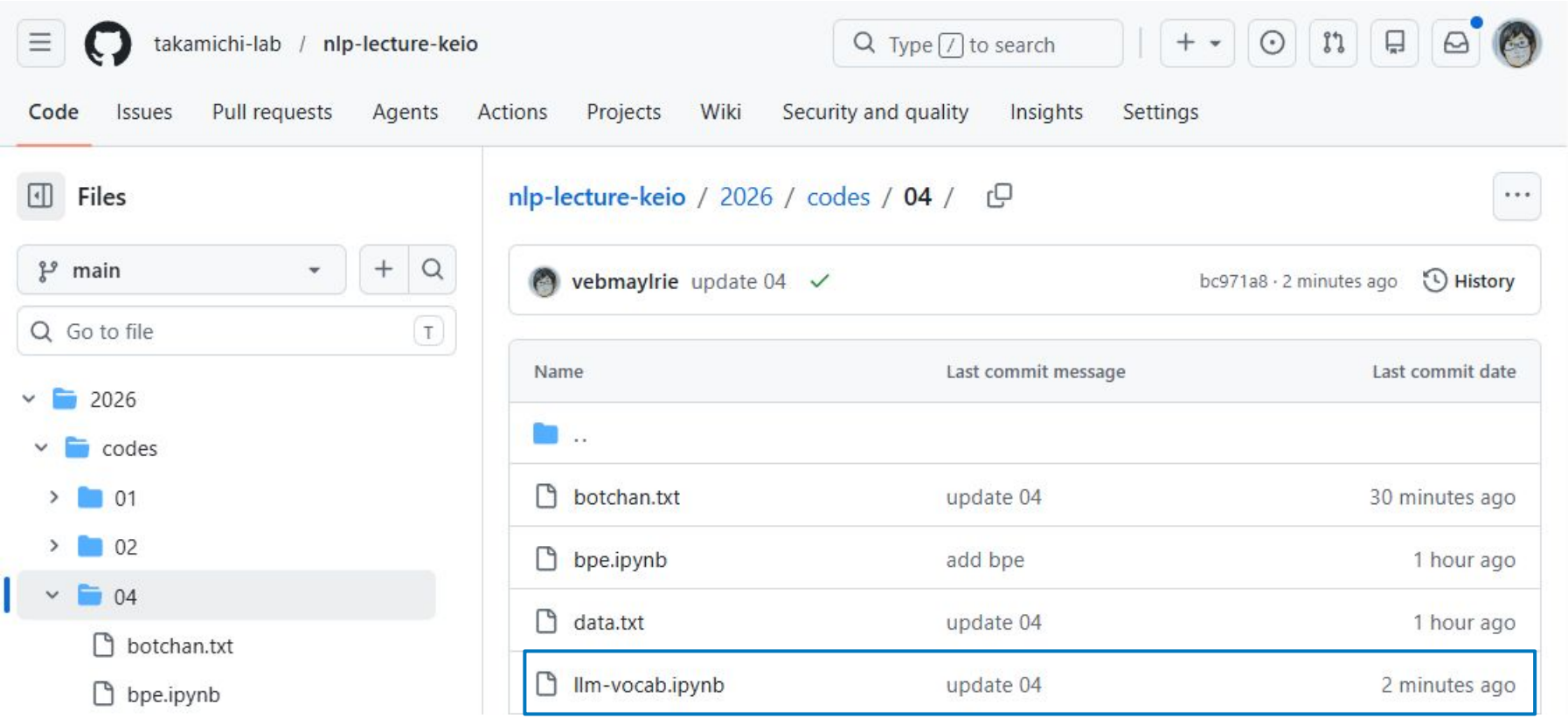
大規模言語モデル (LLM) とサブワード

LLM はサブワード語彙を用いている．



大規模言語モデル (LLM) の語彙を覗いてみよう

https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/04



まとめ

まとめ

- 単語埋め込み (word embedding)
  - 単語を計算機で扱うために数値表現する方法

- Word2Vec
  - 分布仮説に基づいて，隣接単語から当該単語の埋め込みを求める

- サブワード
  - 語彙爆発を避けるための語彙獲得法で，LLM における主流

課題 (どれか 1 つを選択し提出せよ)

候補 1: Word2Vec

新たなコーパスを用いて Word2Vec モデルを学習し得られた埋め込みのアナロジーについて考察せよ．

⚠ 注意：GPU環境が必須

候補 2: SentencePiece

新たなコーパスを用いて SentencePiece モデルを学習し，得られたサブワード語彙について考察せよ．

GPUは不要．

候補 3: 学習済みモデル

学習済みの単語埋め込みモデルの埋め込みについて考察せよ．

考察ポイント:

- アナロジー
- 言語差
- 性差，etc.

GPU利用が好ましい．